Symbolic Arguments in Rotor

How unconstrained argv lets a model checker discover triggering inputs

Rotor · BTOR2 · btormc · RISC-V

Jasmin Begic · Daniel Wassie · University of Salzburg

Concrete vs Symbolic Arguments

The key difference: who picks the input — you, or the solver?

Concrete run

You pick the input.

The program runs with exactly those bytes.

If the bug hides behind a specific argv, you must already know it.

```
$ ./program HELLO
  argv[1] = "HELLO"
  exit code: 0   (no crash)

$ ./program CRASH
  argv[1] = "CRASH"
  exit code: 1   (bug!)

# You had to guess "CRASH".
```

Symbolic run

The solver picks the input.

Each argv byte is an unconstrained bitvector.

btormc searches over ALL possible argv values for one that reaches a bad state.

```
$ btormc prog.btor2
  sat
  b0   (bug reached)
  argv[1][0] = 0x43  ('C')
  argv[1][1] = 0x52  ('R')
  argv[1][2] = 0x41  ('A')
  argv[1][3] = 0x53  ('S')
  argv[1][4] = 0x48  ('H')
```

Why Symbolic argv Matters

Symbolic stdin is not enough. Some bugs are only reachable via command-line input.

Beyond stdin

Many binaries never call read() but still parse argv.

Without symbolic argv, those paths are invisible to the model checker.

Inputs the CPU sees

argv bytes are memory the program actually loads with lb / lw.

Modelling them symbolically lets the solver reason about every dependent branch.

Witness = recipe

A satisfying assignment is not a report — it is the exact argv that triggers the bug.

You can replay it on real hardware.

What changes formally?

Concrete: M runs on one input x → check one final state.

Symbolic: ∃ argv . reachable(M[argv], bad)? — a solver query over the whole input space, bounded by kmax steps.

How Rotor Builds Symbolic argv

The stack is laid out exactly as the SysV RISC-V ABI expects — but argv bytes are unconstrained states.

Rotor initializes the stack segment in four steps:

argv[0] = "prog"
Fixed program name (concrete bytes + null terminator).

argv[1..N] = symbolic states
Each byte is a fresh unconstrained BTOR2 state — the solver picks 0..255.

Pointer array + NULL terminator
Concrete addresses pointing into the string area.

argc + sp
argc is fixed at symbolic_argc + 1; register a0 = argc; sp points to argc.

Stack layout (high → low)

```
argv[1..N] symbolic bytes + '\0'
argv[0] = "prog\0"
argv ptr [0]
argv ptr [1..N]
NULL terminator
argc  ← sp
```

↓ *growing down*

The One-Line Trick: state, not input

BTOR2 forbids inputs in init expressions — so we use an uninitialised state.

An uninitialised **state** in BTOR2 is **unconstrained** — the solver is free to choose any value at step 0 and propagate it through the execution.

That is exactly the semantics we want for a symbolic byte.

Why not just use input?

BTOR2's init operator requires a constant or a state — not an input node.

Using an uninitialised state gives us the same freedom (solver picks any bitvector) while staying inside the grammar.

```
// rotor/src/machine/core.rs (excerpt)

for byte_idx in 0..max_arglen {
    let addr = builder.constd(
        sorts.sid_stack_address, str_addr, None);

    // Unconstrained state = symbolic byte.
    // BTOR2 forbids 'input' in init, so we use
    // an uninitialised state instead.
    let sym_byte = builder.state(
        sorts.sid_byte,
        &format!("argv[{}][{}]",
                 arg_idx + 1, byte_idx),
        Some("symbolic byte".into()),
    );

    current = builder.write(
        sorts.sid_stack_state,
        current, addr, sym_byte, None);
    str_addr += 1;
}
```

Five Test Programs

Each one hides a bug that only triggers on a specific argv — a benchmark for the solver.

#	File	Bug condition	Expected finding
1	test1_crash_string.c	argv[1][0] == 'C' (first byte)	argv[1] = { 0x43, ... }
2	test2_numeric_overflow.c	word(argv[1]) == 0x4142 ('A','B')	argv[1] = { 0x41, 0x42 }
3	test3_length_dependent.c	strlen(argv[1]) == 1 (b0≠0 ∧ b1=0)	any 1-byte non-null string
4	test4_multi_arg.c	argv[1][0]=='X' ∧ argv[2][0]=='Y'	two coupled args (argc=2)
5	test5_checksum.c	byte0 + byte1 == 200	e.g. { 100, 100, ... }

Key point. Tests 2, 4 and 5 cannot be found by symbolic stdin alone — the programs never call read(). Only symbolic argv exposes these branches.

Example: test1_crash_string.c

A minimal program whose exit code depends on one byte of argv[1].

```
uint64_t main(uint64_t argc, uint64_t* argv) {
    uint64_t* arg1;
    uint64_t  first_word, first_byte;

    if (argc > 1) {
        arg1       = (uint64_t*) *(argv + 1);
        first_word = *arg1;

        // extract low byte (no bitwise & in C*)
        first_byte = first_word
                   - (first_word / 256) * 256;

        if (first_byte == 67)   // 'C'
            return 1;            // <- bad exit
    }
    return 0;
}
```

What the solver sees

argc is fixed (= 2).

argv[0] = "prog" (fixed).

argv[1][0..7] = 8 unconstrained bytes.

Bad state: program exits with a non-zero status.

btormc answer: assign argv[1][0] = 0x43 ('C') — all other bytes may be anything.

```
sat
b0
#0  state argv[1][0]  0x43   'C'
#0  state argv[1][1]  0x00   (any)
```

How to Test It Out

Three steps: compile → generate BTOR2 → run the model checker.

1 Compile to RISC-V (selfie C*)

```
$ selfie -c benchmarks/argv-tests/test1_crash_string.c -o test1.m
```

2 Generate BTOR2 with symbolic argv

```
$ cargo run --release -- test1.m \
      --symbolic-argv  --symbolic-argc 1  --max-arglen 8 \
      -o test1.btor2
```

3 Verify with btormc (bounded model check)

```
$ docker run --rm -v "$PWD:/work" btormc \
      btormc -kmax 200 /work/test1.btor2 > test1.wit

$ head -3 test1.wit
sat
b0
```

Reading the Witness

btormc's output tells you exactly which argv bytes make the bad state reachable.

```
sat
b0
#0
0  00000000  initial-stack-base#...
1  01000011  argv[1][0]      0x43   'C'
2  00000000  argv[1][1]      0x00
3  00000000  argv[1][2]      0x00
...
@0
0  <PC, regs, mem ...>
.
```

How to read it

sat — a counterexample exists.

b0 — which bad property was hit.

#0 — initial state assignment (argv bytes here).

@0, @1, … — per-step input assignments.

. — end of trace.

Visualize it. Drop the .wit file into the BTOR2 web visualizer (Part 3) to step through the trace and see which register / memory byte flips at each step.

Recap

Symbolic = unconstrained

Every argv byte becomes an unconstrained BTOR2 state; the solver chooses its value.

Rotor lays out the stack

argv[0], argv[1..N], pointers, argc, sp — all ABI-correct so the binary runs unmodified.

btormc finds the input

A sat witness is the exact argv that drives the program into a bad state.

Try it

5 ready-made test programs in benchmarks/argv-tests/. Compile, generate BTOR2, run btormc.